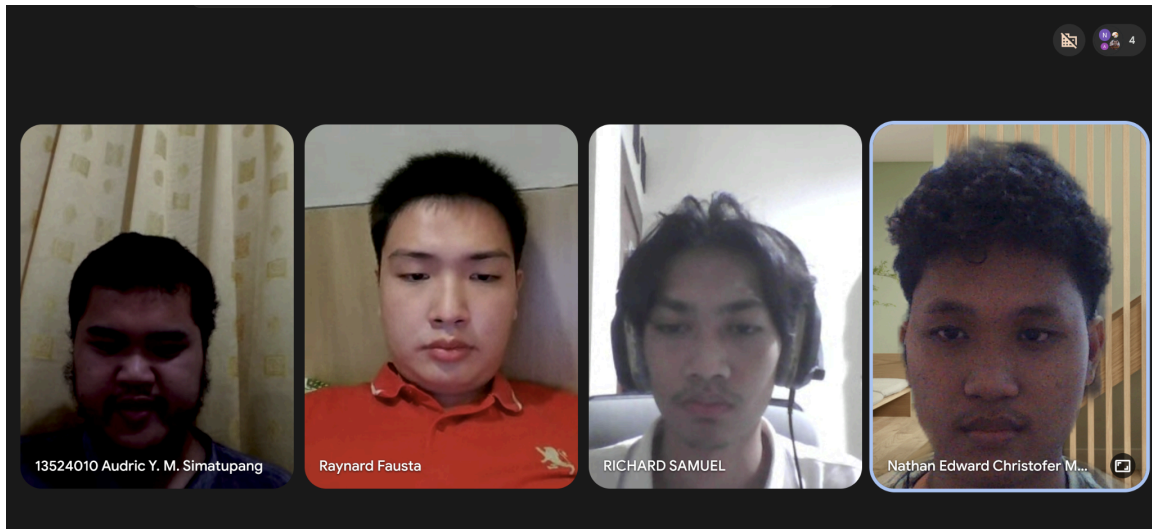


LAPORAN TUGAS BESAR
MATA KULIAH TEORI BAHASA FORMAL DAN OTOMATA
(IF2224)



**TUGAS BESAR MILESTONE 4 : SEMANTIC INTERMEDIATE CODE AND
INTERPRETER**

KELOMPOK SHT-K02

Anggota :

13524010	-	Audric Yusuf Maynard Simatupang
13524052	-	Raynard Fausta
13524062	-	Nathan Edward Christofer Marpaung
13524112	-	Richard Samuel Simanullang

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

DAFTAR ISI

LAPORAN TUGAS BESAR.....	1
DAFTAR ISI.....	2
Teori Singkat.....	0
Semantic Analysis.....	0
Abstract Syntax Tree (AST).....	0
Perancangan & Implementasi.....	2
Perancangan.....	2
Implementasi.....	3
AST_Builder.....	3
Struktur Data Symbol_Table.....	3
AST_Decorator.....	3
Pengujian.....	3
Testing 1.....	3
Kesimpulan dan Saran.....	4
Kesimpulan.....	4
Saran.....	4
Lampiran.....	5
Link Repositori.....	5
Pembagian Tugas.....	5
Referensi.....	6

1. Penjelasan Konsep

Intermediate Code

Intermediate Code adalah representasi dari program yang sudah disimplifikasi untuk mengurangi kompleksitas komputasi dalam source code program awal. Dalam pembuatan compiler, pembuatan intermediate code adalah tahap terakhir dimana daftar instruksi yang rumit di dalam source code ataupun AST diubah menjadi proses sekuensial yang mirip seperti bahasa mesin namun masih mudah dipahami. Simplifikasi ini akan memudahkan komputasi sehingga compiler akan mengikuti daftar instruksi yang sudah disimplifikasi tersebut untuk menjalankan operasi dibandingkan menjalankan operasi source code yang kompleks secara keseluruhan.

Three-Address Code

Three-Address Code adalah salah satu format intermediate code dimana setiap instruksi hanya terdiri oleh 3 alamat secara maksimal. Setiap instruksi hanya terdiri dari 2 operand dan 1 hasil secara maksimal. Three-Address Code ini akan memiliki instruksi memori komputer yang menyerupai alokasi alamat memori di dalam komputer, sehingga kita dapat menggunakan konsep struktur memori Stack untuk membangun hasil instruksi dari Intermediate Code yang dibuat.

Virtual Machine dan Eksekusi Program

Virtual Machine adalah emulasi komputer fisik berbasis software. Virtual Machine berjalan seperti sebuah komputer fisik yang memiliki kekuatan komputasi CPU, memori, dan storage tersendiri untuk menjalankan sebuah program. Dalam konteks compiler dan interpreter, virtual machine adalah jembatan untuk mengeksekusi deretan instruksi dari intermediate code dan memberi hasil dari eksekusi instruksi tersebut. Namun, untuk melakukan eksekusi instruksi dan menyimpan hasil eksekusi, sebuah virtual machine membutuhkan suatu memori. Dalam tugas ini, eksekusi akan dilakukan dari pembuatan “virtual machine” yang akan menjalankan Three-Address Code yang sudah dibuat,

dimana penyimpanan memori akan diimplementasikan berdasarkan konsep memori Stack Register.

Stack Register

Stack Register adalah arsitektur register yang menyimpan informasi eksekusi kode di dalam sebuah struktur data stack. Stack register terdiri dari tumpukan alamat memori yang menyimpan suatu instruksi data. Setiap pemanggilan fungsi yang baru akan disimpan keseluruhan frame program di dalam stack, dimana alamat memori instruksi kode dalam fungsi tersebut akan disimpan masing-masing di dalam stack tersebut seperti sebuah tumpukan buku. Eksekusi program akan berjalan berdasarkan memori yang disimpan di dalam stack, dimana program tinggal membaca instruksi dari atas stack dan menghapus tumpukan instruksi kode (pop) yang sudah dibaca tersebut. Ketika program selesai, stack akan bertemu dengan alamat `ret_address` yang akan mengembalikan hasil fungsi dan mengembalikan pointer memori ke frame sebelumnya. Dalam konteks tugas besar ini, kita dapat menggunakan arsitektur yang sama untuk membangun stack yang akan menyimpan Three-Address Code dan melakukan eksekusi berdasarkan stack yang dibuat tersebut.

Untuk memastikan tidak ada pembacaan atau penyimpanan kode yang salah di dalam stack, pengecekan terhadap data yang akan diproses dilakukan selama eksekusi program. Jika language yang didesain bersifat weakly-typed (mudah untuk mengubah tipe data), Interpreter yang didesain harus melakukan pengecekan terhadap tipe data antara Decorated AST yang sudah dibuat dengan kondisi data dalam stack secara runtime. Jika terdapat perbedaan dalam tipe data, maka program harus mengirim `RuntimeError` untuk menandakan adanya error perubahan tipe data pada program.

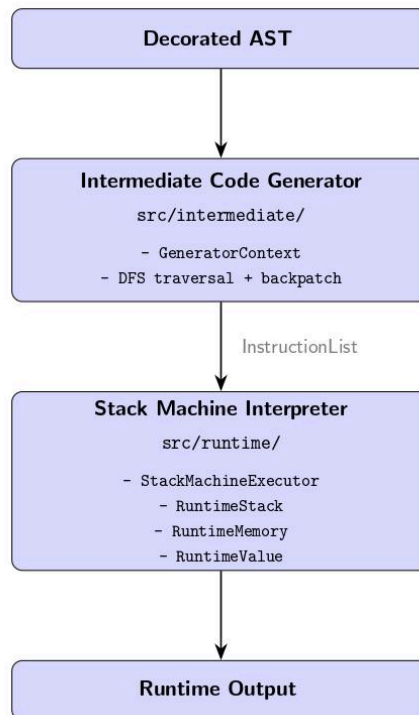
Selain itu, kita juga dapat menetapkan pengecekan tambahan pada kondisi stack yang sudah dibuat. Kita dapat memberikan alokasi batas memori pada stack, dimana stack akan mengalami error `StackOverflow` jika stack terus menerus melakukan push data dan memori pada dirinya hingga melewati batas memori.

Stack juga dapat didesain untuk memberi error `StackUnderflow` jika stack melakukan pop ketika stack berkondisi empty (tidak memiliki penyimpanan data apapun). Jika terdapat eksekusi program yang melampaui batas alamat dan melakukan overwrite pada alamat memori instruksi program lainnya, maka stack dapat mengirim error `StackSmashing` untuk menandakan adanya overwrite alamat memori secara ilegal. Kita juga dapat memastikan bahwa jumlah push alamat memori yang dilakukan simetris dengan jumlah pop di dalam stack, dan mengirim error `StackCorruption` jika terdapat perbedaan antara jumlah pop dan push sehingga terdapat memori sampah yang tinggal di dalam stack. Beberapa error lainnya yang dapat diimplementasikan dalam pengecekan eksekusi program adalah `IndexOutOfBoundsException` (mengakses indeks array diluar batas), `OverflowError` dan `UnderflowError` (operasi terhadap numerik yang membuat besar numerik melewati batas bit), dan `InvalidJumpTarget` (Stack melompat ke daftar instruksi yang invalid).

2. Perancangan & Implementasi

A. Rancangan Arsitektur

Pada Milestone 4 ini ditambahkan lapisan backend compiler Arion untuk melengkapi Lexical Analysis, Syntax Analysis dan Semantic Analysis. Perubahan yang terjadi ini terdiri dari dua komponen utama yang bekerja secara berurutan: Intermediate Code Generator yang mengubah Decorated AST menjadi daftar instruksi sekuensial, dan Stack Machine Interpreter yang mengeksekusi instruksi tersebut. Kedua komponen diimplementasikan dalam bahasa C dan ditempatkan di dua direktori terpisah, `src/intermediate/` dan `src/runtime/`, mengikuti prinsip pemisahan tanggung jawab. Berikut gambaran penambahan yang dilakukan :



B. Implementasi Intermediate Code Generator

Intermediate Code Generator adalah komponen Back-end pertama yang menerima Decorated AST dari Semantic Analyzer dan menghasilkan InstructionList dan daftar instruksi sekuensial berbasis stack machine. Modul ini sepenuhnya diimplementasikan dalam dua file: `intermediate_code.c` dan `intermediate_code_generator.c`.

1. Struktur Data Instruksi

Setiap instruksi IC direpresentasikan oleh struct Instruction yang memiliki empat field yaitu :

- Opcode berisi jenis instruksi
- Level berisi perbedaan lexical level untuk akses variabel non-lokal
- Operand sebagai alamat memori, nilai literal, ataupun sub OPR

Kumpulan instruksi disimpan dalam InstructionList — array dinamis yang kapasitasnya berkembang otomatis saat penuh. Selain append (instructionListEmit), tersedia pula instructionListPatchOperand untuk mengubah operand instruksi yang sudah ter-emit, yang menjadi fondasi mekanisme backpatching.

2. Tabel Instruksi (Opcode)

Instruksi OPR menggunakan field operand sebagai sub-kode yang menentukan operasi spesifik:

Opcode	Deskripsi	Efek pada Stack
INT n	Alokasikan n sel memori (global saat awal, lokal saat CAL)	—
LIT v	Push nilai literal v ke stack	push(v)
LOD lv a	Load nilai dari memori alamat a, level lv, ke stack	push(mem[a])
RLOD lv a	Mirip LOD, tapi digunakan khusus untuk baca slot return value	push(mem[a])
STO lv a	Pop nilai dari stack, simpan ke memori alamat a, level lv	pop → mem[a]
CAL tb a	Panggil subprogram pada baris a (tab index tb untuk frame)	setup frame
JMP a	Unconditional jump ke baris instruksi a	—
JPC a	Pop nilai; jika 0 (false), jump ke baris a	pop → PC?

OPR k	Jalankan sub-operasi k (aritmetika, relasional, I/O, dsb.)	pop(s), push(hasil)
RET	Kembali dari subprogram, pulihkan konteks frame	restore frame

dsadas

Kode	Operasi	Kode	Operasi
1 – NEG	Negasi unary	11 – GTR	Lebih dari (>)
2 – ADD	Penjumlahan	12 – LEQ	Kurang dari sama dengan (<=)
3 – SUB	Pengurangan	13 – WRT	Cetak nilai (write)
4 – MUL	Perkalian	14 – WRTLN	Cetak nilai + newline (writeln)
5 – DIV	Pembagian integer	15 – RDIV	Pembagian real
6 – MOD	Modulo	16 – TO_REAL	Konversi integer ke real
7 – EQL	Sama dengan (=)	17 – INDEX_ERR OR	Error batas array (runtime)
8 – NEQ	Tidak sama dengan (<>)	18 – RANGE_ERR OR	Error subrange (runtime)
9 – LSS	Kurang dari (<)	19 – TO_CHAR	Konversi integer ke char
10 – GEQ	Lebih dari sama dengan (>=)	20 - POP	Buang nilai teratas stack

3. Strategi Generate Kode

Generator menelusuri Decorated AST secara rekursif dengan pendekatan DFS (Depth-First Search) yang perlu dievaluasi sebelum hasilnya digunakan. Seluruh state generation diatur dalam struct internal GeneratorContext yang diteruskan melalui setiap pemanggilan fungsi rekursif, sehingga semua fungsi generate* berbagi state yang sama tanpa menggunakan variabel global.

4. Mekanisme Backpatch

Karena Intermediate Code dihasilkan dalam satu pass linear, target alamat jump sering belum diketahui saat instruksi JMP/JPC

pertama kali di-emit. Implementasi mengatasi ini dengan pola berikut:

- `emitPatchable()` meng-emit JMP atau JPC dengan operand sementara 0, dan mencatat indeks instruksi tersebut ke variabel `line`.
- Setelah kode blok target selesai di-emit, diisi operand yang benar.
- Untuk forward call subprogram, `emitCall()` menyimpan instruksi CAL ke `pendingCalls`. Setelah semua subprogram ter-generate, `patchPendingCalls()` memproses seluruh daftar.

Pola ini digunakan secara konsisten untuk IF-THEN-ELSE, WHILE, FOR, REPEAT-UNTIL, CASE, dan pemanggilan subprogram ke depan.

C. Implementasi Interpreter & Stack Machine

Interpreter bertindak sebagai virtual machine yang membaca `InstructionList` dan menjalankannya satu per satu. Komponen ini terdiri dari empat modul dengan lapisan tanggung jawab yang jelas

1. Arsitektur Modul

Interpreter terdiri dari empat modul yang memiliki lapisan tanggung jawab yang jelas yaitu:

- Modul `runtime_value` memiliki tanggung jawab representasikan satu nilai bertipe dinamis
- Modul `runtime_memory` memiliki tanggung jawab menyimpan variabel global
- Modul `runtime_stack` memiliki tanggung jawab sebagai stack untuk operand serta frame management
- Modul `stack_machine_executor` memiliki tanggung jawab sebagai mesin virtual

2. RuntimeValue : Tipe Data Runtime

Nilai runtime direpresentasikan sebagai tagged union yang mendukung enam tipe. Nilai numerik selalu menyimpan representasi long long dan double secara bersamaan, sehingga operasi perbandingan dan konversi antar tipe numerik dapat dilakukan tanpa percabangan tipe yang berulang. Tipe STRING menggunakan heap-allocated string dengan manajemen memori eksplisit melalui `runtimeValueCopy()` dan `runtimeValueFree()`.

3. Runtime Stack : Operand

Stack diimplementasikan sebagai array dinamis yang tumbuh dua kali lipat kapasitasnya saat penuh, dengan batas atas maxSize yang diperiksa pada setiap Push.

Operasi	Deskripsi
Push / Pop	Operasi LIFO standar dengan validasi batas
Peek	Baca elemen puncak tanpa menghapus
GetAt / SetAt	Akses acak berindeks — digunakan executor untuk membaca/menulis variabel lokal dalam frame tanpa membongkar stack
Truncate	Potong stack ke ukuran tertentu saat frame di-pop setelah RET

4. Runtime Memory

Memori global dialokasikan sekali saat instruksi INT pertama dieksekusi menggunakan calloc, menghasilkan array RuntimeValue berukuran tetap. Setiap akses melalui LOD/STO dengan level=0 divalidasi terhadap batas atas sebelum diteruskan.

5. Mekanisme Kerja Interpreter

Interpreter menjalankan program melalui sebuah siklus berulang yang terdiri atas tahap fetch, decode, dan execute. Siklus ini terus berlangsung selama masih terdapat instruksi yang harus diproses.

- Fetch: Interpreter mengambil instruksi berikutnya dari memori program dengan mengacu pada posisi yang ditunjukkan oleh Instruction Pointer (IP).
- Decode: Instruksi yang telah diambil kemudian dianalisis untuk menentukan opcode beserta operand atau parameter yang menyertainya. Proses ini biasanya dilakukan melalui mekanisme pemilihan seperti switch-case atau tabel instruksi.
- Execute: Setelah instruksi dikenali, interpreter menjalankan operasi yang sesuai. Operasi tersebut dapat berupa manipulasi nilai pada stack, perpindahan alur eksekusi dengan memperbarui IP, maupun pengelolaan activation frame selama pemanggilan fungsi.

Ketiga tahap tersebut membentuk siklus utama yang memungkinkan interpreter mengeksekusi program secara bertahap hingga program selesai dijalankan.

6. Validasi Runtime

Interpreter dirancang untuk mendeteksi dan melaporkan pelanggaran runtime tanpa crash, mengikuti model penanganan error bahasa tingkat tinggi. Setiap operasi kritis diperiksa sebelum dieksekusi; jika terjadi pelanggaran, pesan error yang informatif disimpan ke executor->error dan eksekusi dihentikan secara bersih.

Jenis Error	Mekanisme Deteksi	Respons Sistem
Stack Overflow	callDepth >= maxCallDepth (batas 1024 frame) diperiksa setiap CAL	setRuntimeError(), stopped = true
Stack Underflow	count == 0 diperiksa setiap Pop	setRuntimeError(), stopped = true
Stack Corruption	(1) Nilai canary 0x4D344341 diverifikasi saat RET. (2) Pengecekan simetri ukuran stack sebelum Truncate	setRuntimeError(), stopped = true
Out-of-Bounds Access	Setiap LOD/STO divalidasi terhadap batas alamat, baik untuk RuntimeMemory maupun frameSlots	setRuntimeError() dengan alamat yang bermasalah
Array Index Out of Bounds	generateBoundsCheck() meng-emit validasi eksplisit sebelum setiap akses array; jika gagal, OPR INDEX_ERROR dipanggil	Pesan error dengan indeks aktual dan rentang valid [low, high]
Subrange Violation	generateRuntimeRangeCheck() meng-emit validasi sebelum STO ke variabel bertipe subrange	Pesan error dengan nilai aktual dan rentang yang diizinkan
Invalid Jump Target	JMP, JPC, dan CAL memvalidasi $0 \leq \text{target} < \text{jumlah_instruksi}$ sebelum PC diubah	setRuntimeError() dengan PC dan target yang tidak valid
Numerical Overflow/Underflow	Setiap hasil operasi integer dicek terhadap $[-2.147.483.648, 2.147.483.647]$ sebelum di-push	setRuntimeError() Numerical Overflow / Underflow
Division by Zero	DIV dan MOD memeriksa nilai	setRuntimeError() Division by Zero

	pembagi sebelum operasi	
--	----------------------------	--

3. Pengujian

Testing 1

Keterangan Input : Input program Arion dengan assignment nilai ke variabel

Input	
<pre>program TestAssignment; var x: integer; begin x := 10; writeln(x); end.</pre>	
Output	
<pre>=== Intermediate Code === 0 INT 0 7 1 JMP 0 2 2 LIT 0 10 3 STO 0 3 4 LOD 0 3 5 OPR 0 14 6 RET 0 0 === Runtime Output === 10</pre>	
Penjelasan	Pengujian ini membuktikan bahwa assignment tidak lagi hanya dikenali sebagai node pada AST, tetapi sudah benar-benar dijalankan oleh interpreter. Nilai literal 10 harus dimasukkan ke stack, lalu disimpan ke alamat variabel x, kemudian dimuat kembali saat writeln(x) dijalankan.

	Output 10 menunjukkan bahwa instruksi runtime seperti LIT, STO, LOD, dan WRTLN sudah bekerja dengan benar.
--	--

Testing 2

Keterangan Input : Input program Arion dengan ekspresi aritmatika bertingkat

Input
<pre> program TestArithmetic; var a, b, c, result: integer; begin a := 2; b := 3; c := 4; result := (a + b) * c; writeln(result); end.</pre>
Output
<pre> === Intermediate Code === 0 INT 0 10 1 JMP 0 2 2 LIT 0 2 3 STO 0 3 4 LIT 0 3 5 STO 0 4 6 LIT 0 4 7 STO 0 5 8 LOD 0 3</pre>

<pre> 9 LOD 0 4 10 OPR 0 2 11 LOD 0 5 12 OPR 0 4 13 STO 0 6 14 LOD 0 6 15 OPR 0 14 16 RET 0 0 === Runtime Output === 20 </pre>	
Penjelasan	Pengujian ini menunjukkan bahwa ekspresi aritmatika dapat dievaluasi saat runtime. Ekspresi $(a + b) * c$ harus dihitung dengan urutan yang benar, yaitu $2 + 3 = 5$, lalu $5 * 4 = 20$. Output 20 membuktikan bahwa intermediate code generator mampu meratakan ekspresi bertingkat menjadi instruksi sekuensial dan interpreter mampu mengeksekusi operasi ADD dan MUL dengan benar.

Testing 3

Keterangan Input : Input program Arion dengan operasi negasi, tambah, kurang, kali, div, dan mod

Input
<pre> program TestOperators; var a, b: integer; begin a := 17; b := 5; </pre>

```
writeln(-a);  
writeln(a + b);  
writeln(a - b);  
writeln(a * b);  
writeln(a div b);  
writeln(a mod b);  
end.
```

Output

=== Intermediate Code ===

```
0 INT 0 8  
1 JMP 0 2  
2 LIT 0 17  
3 STO 0 3  
4 LIT 0 5  
5 STO 0 4  
6 LOD 0 3  
7 OPR 0 1  
8 OPR 0 14  
9 LOD 0 3  
10 LOD 0 4  
11 OPR 0 2  
12 OPR 0 14  
13 LOD 0 3  
14 LOD 0 4  
15 OPR 0 3  
16 OPR 0 14  
17 LOD 0 3  
18 LOD 0 4  
19 OPR 0 4  
20 OPR 0 14
```

<pre> 21 LOD 0 3 22 LOD 0 4 23 OPR 0 5 24 OPR 0 14 25 LOD 0 3 26 LOD 0 4 27 OPR 0 6 28 OPR 0 14 29 RET 0 0 === Runtime Output === -17 22 12 85 3 2 </pre>	
Penjelasan	Pengujian ini memastikan bahwa berbagai operasi aritmatika yang direpresentasikan sebagai instruksi OPR dapat dijalankan oleh interpreter. Program menguji negasi nilai dengan -a, penjumlahan, pengurangan, perkalian, pembagian integer, dan modulo dalam satu kasus. Output -17, 22, 12, 85, 3, dan 2 menunjukkan bahwa operasi NEG, ADD, SUB, MUL, DIV, dan MOD menghasilkan nilai runtime yang sesuai dan stack machine dapat menangani beberapa ekspresi aritmatika secara berurutan.

Testing 4

Keterangan Input : Input program Arion dengan ekspresi perbandingan boolean

Input

```
program TestComparison;  
var  
    a, b: integer;  
begin  
    a := 10;  
    b := 5;  
    writeln(a > b);  
    writeln(a == b);  
    writeln(a <> b);  
end.
```

Output

=== Intermediate Code ===

```
0 INT 0 8  
1 JMP 0 2  
2 LIT 0 10  
3 STO 0 3  
4 LIT 0 5  
5 STO 0 4  
6 LOD 0 3  
7 LOD 0 4  
8 OPR 0 11  
9 OPR 0 14  
10 LOD 0 3  
11 LOD 0 4  
12 OPR 0 7  
13 OPR 0 14  
14 LOD 0 3  
15 LOD 0 4  
16 OPR 0 8  
17 OPR 0 14
```

<pre>18 RET 0 0 === Runtime Output === true false true</pre>	
Penjelasan	Pengujian ini menunjukkan bahwa ekspresi perbandingan sudah dapat menghasilkan nilai boolean saat runtime. Karena a bernilai 10 dan b bernilai 5, ekspresi <code>a > b</code> bernilai true, <code>a == b</code> bernilai false, dan <code>a != b</code> bernilai true. Output tersebut membuktikan bahwa operasi perbandingan seperti GTR, EQL, dan NEQ sudah dapat dijalankan oleh interpreter, bukan hanya dicek secara semantik.

Testing 5

Keterangan Input : Input program Arion dengan percabangan if-else

Input
<pre>program TestIfElse; var score, result: integer; begin score := 75; if score >= 60 then result := 1 else result := 0; writeln(result); end.</pre>

Output	
<pre> === Intermediate Code === 0 INT 0 8 1 JMP 0 2 2 LIT 0 75 3 STO 0 3 4 LOD 0 3 5 LIT 0 60 6 OPR 0 10 7 JPC 0 11 8 LIT 0 1 9 STO 0 4 10 JMP 0 13 11 LIT 0 0 12 STO 0 4 13 LOD 0 4 14 OPR 0 14 15 RET 0 0 === Runtime Output === 1 </pre>	
Penjelasan	Pengujian ini membuktikan bahwa control flow if-else sudah dapat dieksekusi. Kondisi <code>score >= 60</code> bernilai true, sehingga interpreter harus menjalankan cabang then dan menyimpan 1 ke variabel result. Output 1 menunjukkan bahwa hasil perbandingan dapat digunakan oleh instruksi conditional jump, dan hanya cabang yang benar yang dijalankan.

Testing 6

Keterangan Input : Input program Arion dengan perulangan while

Input
<pre> program TestWhile; var i: integer; begin i := 3; while i > 0 do begin writeln(i); i := i - 1; end; end. </pre>
Output
<pre> === Intermediate Code === 0 INT 0 7 1 JMP 0 2 2 LIT 0 3 3 STO 0 3 4 LOD 0 3 5 LIT 0 0 6 OPR 0 11 7 JPC 0 15 8 LOD 0 3 9 OPR 0 14 10 LOD 0 3 11 LIT 0 1 12 OPR 0 3 13 STO 0 3 14 JMP 0 4 </pre>

<pre>15 RET 0 0</pre> === Runtime Output === <pre>3 2 1</pre>	
Penjelasan	Pengujian ini menunjukkan bahwa perulangan while sudah dapat dijalankan menggunakan mekanisme jump pada intermediate code. Nilai i dimulai dari 3, lalu dicetak dan dikurangi satu pada setiap iterasi. Output 3, 2, dan 1 membuktikan bahwa kondisi loop dievaluasi ulang, body loop dijalankan berulang, nilai variabel diperbarui, dan interpreter keluar dari loop ketika kondisi $i > 0$ sudah false.

Testing 7

Keterangan Input : Input program Arion dengan update variabel secara berulang

Input
<pre>program TestMemoryPersistence; var i, total: integer; begin i := 1; total := 0; while i <= 5 do begin total := total + i; i := i + 1; end; writeln(total);</pre>

end.	
Output	
<pre> === Intermediate Code === 0 INT 0 8 1 JMP 0 2 2 LIT 0 1 3 STO 0 3 4 LIT 0 0 5 STO 0 4 6 LOD 0 3 7 LIT 0 5 8 OPR 0 12 9 JPC 0 19 10 LOD 0 4 11 LOD 0 3 12 OPR 0 2 13 STO 0 4 14 LOD 0 3 15 LIT 0 1 16 OPR 0 2 17 STO 0 3 18 JMP 0 6 19 LOD 0 4 20 OPR 0 14 21 RET 0 0 === Runtime Output === 15 </pre>	
Penjelasan	Pengujian ini fokus pada kemampuan stack memory untuk

	mempertahankan dan memperbarui nilai variabel selama program berjalan. Variabel total terus diperbarui di dalam loop dengan menambahkan nilai i dari 1 sampai 5. Output 15 menunjukkan bahwa nilai variabel tidak hilang antar-instruksi, operasi LOD dan STO berjalan konsisten, dan interpreter dapat mengelola state program sepanjang beberapa iterasi.
--	---

Testing 8

Keterangan Input : Input program Arion dengan beberapa pemanggilan writeln

Input
<pre> program TestWriteln; var x, y: integer; begin x := 4; y := 6; writeln(x); writeln(y); writeln(x + y); end.</pre>
Output
<pre> === Intermediate Code === 0 INT 0 8 1 JMP 0 2 2 LIT 0 4 3 STO 0 3 4 LIT 0 6</pre>

```

5 STO 0 4
6 LOD 0 3
7 OPR 0 14
8 LOD 0 4
9 OPR 0 14
10 LOD 0 3
11 LOD 0 4
12 OPR 0 2
13 OPR 0 14
14 RET 0 0

```

```

=== Runtime Output ===

```

```

4
6
10

```

Penjelasan

Pengujian ini menekankan fitur output akhir dari interpreter. Program mencetak dua nilai variabel dan satu hasil ekspresi aritmatika. Output 4, 6, dan 10 menunjukkan bahwa writeln tidak hanya muncul sebagai procedure call pada AST, tetapi sudah diterjemahkan menjadi instruksi output dan benar-benar mencetak nilai hasil eksekusi program.

4. Kesimpulan dan Saran

Kesimpulan

Milestone 4 berhasil mengimplementasikan dua komponen backend compiler Arion yang melengkapi pipeline kompilasi dari Milestone 1–3: Intermediate Code Generator dan Stack Machine Interpreter. Dengan selesainya milestone ini, compiler Arion mampu menjalankan program secara penuh.

Intermediate Code Generator berhasil konversi Decorated AST jadi InstructionList melalui traversal DFS rekursif dengan mekanisme backpatching satu pass. Generator mendukung seluruh konstruksi bahasa Arion yang ditetapkan dalam spesifikasi, mencakup ekspresi aritmatika dan logika, percabangan IF-THEN-ELSE, perulangan WHILE, FOR, dan REPEAT-UNTIL, pernyataan CASE, serta pemanggilan prosedur dan fungsi bersarang dengan passing parameter.

Stack Machine Interpreter berhasil mengeksekusi InstructionList melalui siklus Fetch-Decode-Execute dengan model memori yang terdiri dari RuntimeMemory untuk variabel global dan RuntimeStack untuk operand serta frame management. Sebagai fitur bonus, interpreter dilengkapi dengan validasi runtime yang mendeteksi dan melaporkan sepuluh jenis pelanggaran.

Saran

Pengujian otomatis berbasis file (input menjadi expected output) akan sangat membantu pada milestone berikutnya mengingat jumlah kasus uji yang perlu dicakup semakin besar.

5. Lampiran

Link Repositori

<https://github.com/RayapSunggal/SHT-Tubes-IF2224-2026>

Pembagian Tugas

Nama	Tugas	Persentase
Audric Y. M.Simatupang - 13524010	Pembuatan hasil test case pada laporan, Penyusunan laporan, Pembuatan Intermediate Code Generator	25%
Raynard Fausta - 13524052	Bugfixing source code, Testing source code	25%
Nathan E.C Marpaung - 13524062	Pembuatan stack machine executor	25%
Richard S Simanullang -	Bugfixing source code, Testing source	25%

13524112	code	
----------	------	--

6. Referensi

